

KG-SFuzz: Knowledge-Graph Empowered State-Aware Device Driver Fuzzing

Anonymous IEEE S&P submission
Paper ID 1180

Abstract—Device drivers are security-critical, stateful components typically implemented as complex state machines with deep execution paths governed by strict transition sequences. Existing fuzzing techniques struggle to model such intricate stateful behaviors, especially in capturing valid cross-interface transitions. While state-aware fuzzers infer interface dependencies through either static trace analysis or dynamic runtime feedback, they often lack semantic understanding of state transition, resulting in inefficient exploration and limited penetration into deeply nested code paths.

This paper presents KG-SFuzz, a knowledge-guided, state-aware fuzzing framework for systematic exploration of device drivers. KG-SFuzz introduces the API State Dependency Graph (ASDG), a semantic data model that captures state transitions, data flows, and parameter constraints by combining large language model (LLM)-based reasoning with static analysis of driver source code. Guided by the ASDG, KG-SFuzz synthesizes context-aware fuzz drivers, employs a Coverage-Distance Collaborative Seed Scheduler to balance coverage-guided exploration with state-driven progression, and continuously refines the ASDG using in-loop execution feedback. The evolving ASDG further drives adaptive mutation and advanced fuzz driver synthesis, enabling complex and semantically valid test scenarios. We evaluate KG-SFuzz on real-world device drivers from the Samsung Galaxy A54 and the Linux kernel 6.7. Results show that KG-SFuzz achieves higher code coverage and detects more vulnerabilities than prior approaches. Notably, KG-SFuzz uncovers 15 previously unknown vulnerabilities, including 9 eligible for CVE assignment, demonstrating its effectiveness in exposing deep-seated flaws in complex driver code.

Index Terms—Fuzzing, Operating System Security, Vulnerability Detection, LLMs

I. INTRODUCTION

Device drivers act as fundamental intermediaries between hardware components and operating systems. Prior approaches [1], [2], [3] have shown that drivers are disproportionately responsible for kernel-level vulnerabilities. Fuzzing testing [4], [5], [6], [7], [8], [9], [10], [11] has achieved notable success in discovering kernel vulnerabilities. However, existing driver fuzzing approaches lack deep semantic understanding of stateful behavior and inter-API dependencies, which severely hinders their ability to explore deep, state-dependent code paths. The root of this challenge lies in the inherent logic complexity of device driver.

To address this issue, recent research [12], [13], [14], [15] have focused on modeling driver states to enhance the efficiency of system testing and vulnerability discovery. However, they suffer from input bias, semantic sparsity, and modeling long-range dependencies [16], making it highly challenging to infer precise state constraints from raw system call sequences.

With the development of AI technologies, LLMs have been employed to infer API semantics and generate Syzkaller descriptions for device drivers. However, these methods [17] suffer from hallucination and inconsistency in LLMs, and lack expressive power to capture implicit state transitions and temporal dependencies.

Furthermore, even when state information is partially captured, a critical gap remains in systematic exploration of state machine. StateFuzz [18] introduces state variable feedback to guide exploration. Although this improves coverage over basic coverage-only strategies, it still lacks semantic precision in modeling states evolution and APIs interactions across different states. Moreover, it largely relies on the underlying fuzzer (Syzkaller) to perform state traversal without explicit guidance on valid transition paths or coverage criteria for states. The state transitions remain implicitly inferred and heuristically driven, leading to inefficient, non-deterministic exploration that often fails to reach deep vulnerabilities.

This paper presents KG-SFuzz, a novel knowledge-driven, state-aware fuzzing framework that incorporates semantic knowledge into the fuzzing pipeline for device drivers. KG-SFuzz constructs an ASDG that captures state transitions, data flows, and parameter constraints among driver's syscall interfaces. Unlike prior LLM-based approaches that directly generate fuzzing descriptions, KG-SFuzz combines static program analysis with LLMs-based inference to recover high-level behavioral relations among interfaces. Specifically, KG-SFuzz first extracts code facts (e.g., types, macros, and call chains) from driver source code through static program analysis. It then employs an LLM-based reasoning framework that iteratively summarizes API behaviors, retrieves missing context from the extracted code facts, and infers state transitions, data flows and parameter dependencies.

The constructed ASDG enables the fuzzer to generate semantically state-aware fuzz drivers, significantly improving coverage and bug detection in device driver testing. KG-SFuzz integrates a coverage-distance collaborative seed scheduler. Distance-driven Fuzz (D-Fuzz) performs targeted exploration toward specific goal states by leveraging distance instrumentation. Coverage-driven Fuzz (C-Fuzz) systematically explores alternative execution paths to maximize structural coverage, adhering to state constraints encoded in the graph. Crucially, the ASDG evolves continuously based on the feedback from in-loop execution, allowing KG-SFuzz to improve its understanding of program behavior over time. This evolving graph improves the precision of fuzz driver synthesis and mutation

strategies: (1) context-aware fuzz driver generation that enable inputs to comply with the evolving semantic constraints, and (2) adaptive mutation guided by high-level execution patterns (e.g., new API sequences and parameter bindings). Collectively, these mechanisms empower KG-SFuzz to synthesize complex, realistic test cases that traverse long, cross-state execution paths. Where conventional fuzzers typically struggle.

We evaluate KG-SFuzz on eight real-world Linux device drivers, including general-purpose drivers (Linux-6.7) and proprietary drivers (Galaxy A54 platform). On average, KG-SFuzz outperforms Syzkaller by **4.92%**, SyzDescribe by **131.24%**, and StateFuzz by **11.46%** in code coverage on the Linux-6.7 drivers. For Galaxy A54 drivers, KG-SFuzz achieves a **32.57%** coverage improvement over Syzkaller under identical time constraints. Additionally, KG-SFuzz uncovered 15 previously unknown vulnerabilities, nine of which qualified for CVE assignment.

In summary, our contributions are as follows:

- 1) We propose KG-SFuzz, the first fuzzer that integrates LLM-based reasoning with static analysis to construct a semantically rich ASDG for device drivers, enabling precise modeling of state transitions, data flows, and parameter constraints.
- 2) We design a coverage-distance collaborative seed scheduler that unifies coverage-driven (C-Fuzz) and state-distance-guided exploration (D-Fuzz) for deterministic, goal-directed, and coverage-aware exploration, significantly improving transition efficiency, data flow and state reachability.
- 3) We propose a dynamic knowledge evolution mechanism that updates the ASDG with runtime feedback, enabling adaptive fuzz driver synthesis and knowledge-guided mutation for uncovering deep, cross-state bugs.
- 4) We implement and evaluate KG-SFuzz on real-world drivers, demonstrating superior performance over state-of-the-art fuzzers and uncovering 15 new vulnerabilities with 9 eligible for CVEs assignment. These result highlight the practical impact of semantics-enhanced, knowledge-driven fuzzing.

Our work establishes a new paradigm for intelligent fuzzing, where structured knowledge graphs, powered by LLMs and program analysis, enable the systematic and deep exploration of complex, stateful software system.

II. BACKGROUND

A. Device Driver

Device drivers, the critical privileged attack surfaces in the Linux kernel [19], [20], are low-level software components responsible for managing hardware-device interactions within operating systems. Due to their privileged execution context in kernel space, vulnerabilities in drivers can lead to system crashes, privilege escalation, or complete system compromise [21], [22]. Modern drivers are inherently stateful, meaning their behavior depends on an internal state determined by prior operations such as initialization, configuration, and resource

allocation. These states are often governed by strict sequences of system calls.

Furthermore, driver APIs frequently impose intricate parameter constraints, requiring specific fields within input structures to remain consistent across multiple calls due to both explicit and implicit inter-dependencies. These semantic and stateful requirements render traditional, state-agnostic fuzzing techniques ineffective, as they frequently fail to generate valid driver interaction sequences or respect cross-call parameter invariants.

B. Coverage-Guided Fuzzing

Coverage-guided fuzzers [23], [4], [24], [25], [26], [27], [28], [29] have achieved remarkable success in vulnerability discovery by using runtime feedback to guide input evolution. However, they operate largely at the syntactic level and struggle with semantic validity in stateful environments. In particular:

- **Shallow exploration:** Without understanding state pre-conditions, these fuzzers rarely progress beyond early-stage API calls, failing to trigger deep, multi-step execution paths.
- **Inefficient mutation:** Random mutations often produce invalid sequences that are rejected at entry points, leading to stagnation in coverage gains.
- **No long-range dependency modeling:** Even if individual calls are valid, their composition may violate protocol semantics, making it difficult to reach critical code guarded by multiple state checks.

While Syzkaller supports basic state modeling through hand-written or learned descriptions, its DSL (syzlang) lacks native constructs for expressing temporal relationships or state transition guards, limiting expressiveness and precision.

C. State-aware Fuzzing

To overcome these limitations, recent works have explored state-aware fuzzing techniques. IMF [12] and Moonshine [13] analyze execution traces from real workloads to infer API dependencies. While effective in capturing observed usage patterns, they suffer from input bias, if a valid transition is missing in the trace, it will not be modeled. Moreover, raw traces lack semantic annotations, making it hard to distinguish between causal dependencies and co-occurrence. Healer [14] and MOCK [15] use runtime feedback to learn valid sequences incrementally. However, they rely on stochastic exploration and often fail to capture long-range transitions due to the combinatorial explosion of possible API sequences.

Recently, LLMs have been applied to assist fuzzing, enabling the capture and generation of input patterns that are syntactically correct, semantically rich, and consistent with functional intent and contextual requirements. PromptFuzz [30], CKGFuzzer [31] have demonstrated significant success in applying LLM-based fuzzing techniques to application-level testing. Building on these advances, KernelGPT [17] pioneers the integration of LLMs into kernel fuzzing. It employs LLMs to generate syzlang code conforming to Syzkaller's

system call specifications, and employs an iterative, multi-stage prompting strategy combined with Syzkaller’s verification feedback to automatically repair `syzlang` specifications, thereby enhancing Syzkaller’s efficiency. While promising, face two critical limitations. First, Syzkaller’s domain-specific language (`syzlang`) lacks expressive power to capture implicit state transitions and temporal dependencies. Second, LLMs alone are prone to hallucination when reasoning over long API sequences. Prior works that feed raw code or API signatures into LLMs often produce semantically inconsistent or invalid call sequences, undermining their utility in practical fuzzing. Additionally, `StateFuzz` adopts a finer-grained approach: it statically identifies key state variables in driver code and dynamically tracks their value changes during execution, using this information to guide input generation toward unexplored or under-explored state configurations.

These techniques can unlock some previously inaccessible execution paths and increase the likelihood of exposing latent bugs caused by state corruption or improper state management. However, they lack a systematic and model-driven exploration methodology that leverages a semantically rich, evolvable model to integrate code structure, API semantics, and runtime dynamics for principled state traversal.

III. OVERVIEW

This section illustrates the challenges in fuzzing stateful device drivers through a real-world example from Samsung Galaxy A54 NPU (Neural Processing Unit) driver, and presents a high-level overview of how `KG-SFuzz` addresses these challenges.

A. Motivation Example

Consider an NPU driver that manages hardware resources through a strict lifecycle, as shown in Table I. To reach deep code such as the `DQBUF`, the fuzzer must execute a sequence of operations: `open()`, `BOOTUP`, `S_GRAPH`, `S_FORMAT`, `STREAM_ON` and `QBUF`, in strict order. Each step establishes a prerequisite state, and violating the sequence leads to immediate rejection in error-checking code.

Suppose a vulnerability exists in `DQBUF`. Triggering it requires not only syntactic correctness but also semantic validity: the driver must have successfully transitioned through prior states and maintained valid internal context. However, existing fuzzers lack the ability to model or reason about this state machine.

As a result, they resort to blind sequence generation, randomly combining system calls and mutating parameters without understanding which transitions are valid or what conditions govern them. This leads to three fundamental challenges.

- **Challenge 1: Lack of semantic state modeling.** Without a structured representation of state transitions and inter-API dependencies, fuzzers cannot distinguish valid from invalid sequences. They repeatedly generate inputs that fail in early checks, such as calling `S_FORMAT` before `BOOTUP` or `DQBUF` before `STREAM_ON`, wasting the

majority of fuzzing effort on semantically meaningless executions.

- **Challenge 2: Energy dispersion due to unguided exploration.** Even when valid prefixes are occasionally generated, the fuzzing energy is uniformly distributed across all APIs and their parameters, regardless of current state context. This results in futile attempts to fuzz deep APIs like `DQBUF` while the system remains in an uninitialized state, preventing systematic progression into deeper code paths.
- **Challenge 3: Static and non-adaptive knowledge.** Existing approaches treat API behavior as fixed, either derived from limited traces or hard-coded rules. They cannot adapt their understanding when new transitions or constraints are discovered during testing, limiting their ability to improve over time.

These challenges collectively prevent conventional fuzzers from achieving deep, deterministic exploration of driver state machines. A new approach is needed to understand driver semantics, guide exploration based on contextual state information, and progressively refine its knowledge during testing.

TABLE I
NPU DRIVER STATE TRANSITIONS

Life Cycle	Interface	State Transition
Initialization	<code>open()</code>	$\emptyset \rightarrow \text{OPEN}$
	<code>BOOTUP</code>	$\text{OPEN} \rightarrow \text{POWER}$
	<code>S_GRAPH</code>	$\text{POWER} \rightarrow \text{GRAPH}$
	<code>S_FORMAT</code>	$\text{GRAPH} \rightarrow \text{FORMAT}$
	<code>STREAM_ON</code>	$\text{FORMAT} \rightarrow \text{STREAMON}$
Execution	<code>QBUF</code>	$\text{STREAMON} \ \& \ \text{QUEUED}$
	<code>DQBUF</code>	$\text{STREAMON} \ \& \ \text{DEQUEUED}$
Finalization	<code>STREAM_OFF</code>	$\text{STREAMON} \rightarrow \text{STREAMOFF}$
	<code>CLOSE</code>	$\forall \rightarrow \text{CLOSE}$

B. Overview of `KG-SFuzz`

`KG-SFuzz` aims to address the above challenges through a knowledge-guided, state-aware fuzzing framework. It integrates semantic modeling, guided scheduling, and dynamic refinement to enable systematic exploration of stateful drivers.

1) **Solution to Challenge 1:** Semantic knowledge extraction and ASDG construction. `KG-SFuzz` begins by analyzing the driver source code using static program analysis to extract code facts such as function signatures, control-flow patterns, and state variable updates. These facts are processed by a LLM-based reasoning engine that infers high-level behavioral semantics. The result is the ASDG, a structured representation where:

- Nodes represent system calls (e.g., `open()`, `S_GRAPH`, `DQBUF`);
- Edges encode state transitions and data dependencies;
- Attributes capture parameter constraints, transition conditions, and usage contexts.

The ASDG makes the implicit state machine explicit, enabling the fuzzer to generate semantically valid sequences and avoid futile paths.

2) **Solution to Challenge 2:** Coverage-Distance Collaborative Scheduling. Guided by the ASDG, KG-SFuzz employs a dual-mode seed scheduling strategy to explore the target API:

- C-Fuzz performs coverage-guided exploration to maximize code coverage of the target API within the current state context.
- D-Fuzz performs distance-guided, goal-oriented exploration, prioritizing seeds that reduce the control-flow distance to target states (e.g., `STREAM_ON`).

The weighted hybrid scheduler integrates both signals to simultaneously promote branch coverage and state progression, enabling sustained and deterministic exploration of deep, state-dependent paths.

3) **Solution to Challenge 3:** Dynamic knowledge evolution. KG-SFuzz continuously refines the ASDG using in-loop execution feedback. When a new transition is successfully triggered or a parameter constraint is observed, the knowledge module updates the graph accordingly. For example, if a specific flag value enables a previously blocked path, this condition is added to the relevant edge. The refined ASDG then guides more precise fuzz driver synthesis and adaptive mutation, forming a semantic feedback loop that improves testing effectiveness over time.

By integrating these components, KG-SFuzz transforms fuzzing from a random, state-agnostic process into a systematic, state-gated exploration. It first stabilizes the driver in prerequisite states, then progressively unlocks and intensifies fuzzing on downstream APIs. This strategy ensures that mutation energy is spent only on reachable, meaningful contexts, enabling deep testing of safety-critical code where conventional fuzzers fall short.

Moreover, after sufficient exploration, KG-SFuzz enhances the ASDG by learning implicit interface dependencies through adaptive mutation. It also leverages the refined graph to generate both malformed but reachable test cases and complex semantic scenarios such as concurrent producer-consumer workloads, enabling deep and realistic driver testing.

IV. METHODOLOGY

The architecture of our proposed approach is depicted in Fig. 1, which comprises three parts: 1) ASDG construction, 2) self-evolving state-aware exploration, and 3) knowledge-guided adaptive exploitation. Below, we introduce each part and explain their interactions.

A. ASDG Construction

We define the ASDG as a comprehensive, directed, and attributed graph that unifies both the *basic facts* extracted from source code and the *semantic information* inferred through LLM reasoning. Fig. 2 is an example of ASDG.

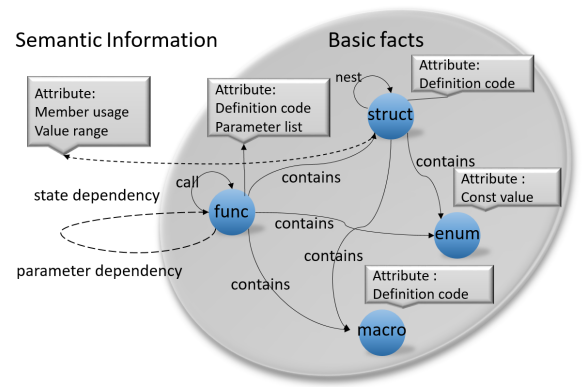


Fig. 2. The API state dependency graph

Formally, an ASDG is denoted as a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A})$, where $\mathcal{V}$ is a set of nodes, $\mathcal{E}$ is a set of directed edges, and $\mathcal{A}$ is a set of attributes assigned to nodes and edges.

The ASDG integrates two complementary layers, **Basic Facts** and **Semantic Information**

1) **Basic Facts:** This layer encodes code entities and their direct structural relationships, extracted by static analysis from the target driver. It represents accurate, reproducible information that serves as the factual foundation for semantic reasoning.

a) **Nodes:** Each node corresponds to a code-level entity:

- **Enumerators nodes**, representing enumeration members annotated with their names and values.
- **Macros nodes**, representing macro constants recorded macro names and textual definitions.
- **Structures nodes**, corresponding to all structure definitions, including the full list of member variables and recursively nested structures.
- **Functions/APIs nodes**, representing both regular functions and target ioctls. Each ioctl node carries its definition, the list of header files it depends on, and detailed parameter information. For structure-type parameters, ASDG recursively records the corresponding structure layout.

b) **Edges:** The edges in the basic layer capture verifiable syntactic or static dependencies:

- **Function-call edges**, connecting functions that invoke one another.
- **Structure-nesting edges**, connecting structures that contain other structures.
- **Reference edges**, connecting macros, enums, and structures when one entity references another.

This layer constitutes the factual foundation of the ASDG, providing all the concrete information required for further reasoning.

2) **Semantic Information:** This layer builds upon the factual graph by incorporating contextual understanding inferred from LLMs. While the basic layer reflects what the code *defines*, the semantic layer captures what the code *implies*. It models latent relations between ioctls, data fields, and states that are not explicitly written in code, but are critical for understanding program behavior.

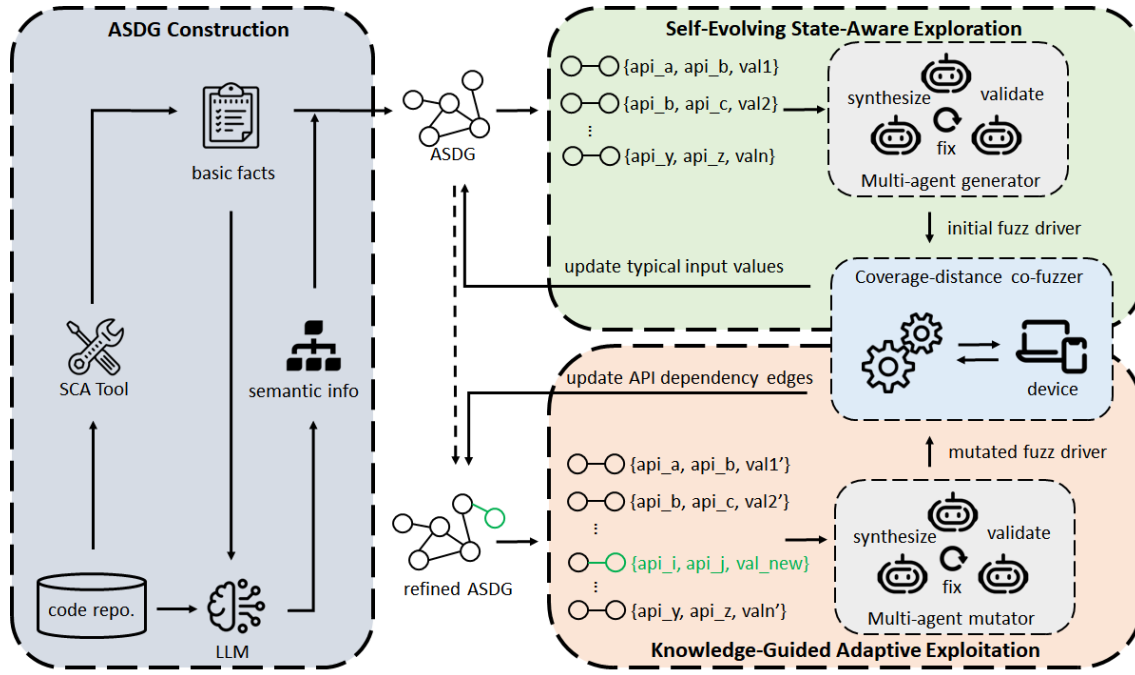


Fig. 1. Overall Architecture of KG-SFuzz

a) *Node-level semantics*: For structures and parameters, LLM inference enriches each node with additional annotations.

- **Value ranges**, estimating the valid or expected numeric intervals of fields (e.g., buffer lengths, counts, or timeout values).
- **Member usage**, describing the functional meaning of fields, such as address, buffer, or flag.

b) *Edge-level semantics*: Edges at the semantic layer capture implicit behavioral dependencies.

- **State dependencies**, describe how one API transitions a shared state that is later consumed by another (e.g., $S_GRAPH \rightarrow S_FORMAT$).
- **Parameter dependencies** specify shared or conditionally related arguments between two ioctls, revealing hidden data-flow or configuration coupling.

Although large language models excel at reasoning and summarization, their application to program analysis is hampered by two inherent problems: **hallucination** and **contextual drift**. Hallucinations occur when the model invents nonexistent behaviors, misreads code semantics, or injects prior knowledge that does not align with the actual source, thereby compromising the trustworthiness of automated reasoning that relies solely on free-form code text.

To infer ioctl dependencies with minimal hallucination, we design a LLMs-based reasoning framework (see algorithm 1), each ioctl is treated as a node whose behavioral semantics are derived through the LLM-based summarizer. For a given reasoning task (e.g., state transition or parameter dependency), the framework first retrieves the ioctl's implementation code and prompts the LLM to summarize its task-relevant functionality. The initial summary is then evaluated

by a verification module to determine whether the reasoning is well-grounded and sufficiently captures the task semantics. If not, the framework automatically identifies the missing contextual code elements, such as enumerations, macros, structure definitions, or helper functions. These retrieved *basic facts* are summarized and reintegrated into the reasoning context, forming an iterative loop of *summarization–evaluation–refinement*.

This design constrains the LLM's reasoning under verifiable static facts, effectively reducing hallucinations and ensuring that each semantic inference remains consistent with the actual code logic. Once all ioctl nodes have been refined into concise, fact-grounded summaries, the framework performs global reasoning over the merged summaries to infer their state and input dependencies. The resulting ASDG captures accurate inter-ioctl relationships, serving as the foundation for downstream analyses such as state-aware fuzzing, where it enables exploration along only valid state transitions and meaningful parameter interactions, thus improving both coverage and bug discovery efficiency.

Through this design, the ASDG achieves a synergistic integration between static analysis and language model reasoning: the static layer ensures factual precision, while the LLM contributes abstraction and semantic depth. This combination allows ASDG to model complex ioctl behaviors and cross-function dependencies with both accuracy and interpretability.

B. Self-Evolving State-Aware Exploration

1) *State-Aware Fuzz Driver Synthesis*: High-quality fuzz drivers are crucial for achieving deep code coverage and exposing critical vulnerabilities. In this chapter, we propose a novel LLM-based state-aware fuzz driver synthesis technique

Algorithm 1 LLMs-based Inference of Task-Specific Relations Among iocltls

Require: Task description T ; set of ioclt names $\mathcal{I}$; Backend interfaces ENUM, STRUCT, MACRO, FUNC; LLM modules Summarizer, Evaluator, MissingCode, Compressor, GraphInfer; maximum refinement rounds K

Ensure: Concise per-ioclt summaries $\{S_i^{\text{final}}\}_{i \in \mathcal{I}}$ and a task-specific relation graph $G = (V, E)$

```

1:  $V \leftarrow \mathcal{I}$ ;  $\mathcal{S} \leftarrow \emptyset$  {Graph nodes are iocltls; collect summaries in  $\mathcal{S}$ }
2: for all  $i \in \mathcal{I}$  do
3:    $C_i \leftarrow \text{Query.func}(i)$  {Primary code for ioclt  $i$ }
4:    $S_i \leftarrow \text{Summarizer}(i, T, C_i)$  {Initial, task-focused code summary}
5:    $r \leftarrow 0$ 
6:   repeat
7:      $(\text{sufficient}, R_i) \leftarrow \text{Evaluator}(S_i, T)$ 
8:     if  $\text{sufficient} = \text{true}$  then
9:       break
10:    else
11:       $\Delta_i \leftarrow \text{MissingCode}(S_i, R_i, T)$ 
12:       $\mathcal{B}_i \leftarrow \emptyset$  {Bag of basic facts and code to fill gaps}
13:      if  $\Delta_i$  requires enumerators then
14:         $\mathcal{B}_i \leftarrow \mathcal{B}_i \cup \text{Query.enum}(\Delta_i)$ 
15:      end if
16:      if  $\Delta_i$  requires macros then
17:         $\mathcal{B}_i \leftarrow \mathcal{B}_i \cup \text{Query.macro}(\Delta_i)$ 
18:      end if
19:      if  $\Delta_i$  requires structs then
20:         $\mathcal{B}_i \leftarrow \mathcal{B}_i \cup \text{Query.struct}(\Delta_i)$ 
21:      end if
22:      if  $\Delta_i$  requires functions then
23:         $\mathcal{B}_i \leftarrow \mathcal{B}_i \cup \text{Query.func}(\Delta_i)$ 
24:      end if
25:       $S_i^{\text{aux}} \leftarrow \text{Summarizer}(\mathcal{B}_i, T)$  {Summarize only the fetched missing pieces}
26:       $S_i \leftarrow \text{Summarizer}(i, T, C_i, S_i^{\text{aux}})$  {Re-summarize ioclt with gap-filling context}
27:       $r \leftarrow r + 1$ 
28:    end if
29:  until  $\text{sufficient} = \text{true}$  or  $r \geq K$ 
30:   $S_i^{\text{final}} \leftarrow \text{Compressor}(S_i)$  {Length-controlled, task-focused summary}
31:   $\mathcal{S} \leftarrow \mathcal{S} \cup \{S_i^{\text{final}}\}$ 
32: end for
33:  $M \leftarrow \text{MERGE}(\mathcal{S})$ 
34:  $G \leftarrow \text{GraphInfer}(M, T)$ 
35: return  $\{S_i^{\text{final}}\}_{i \in \mathcal{I}}$ ,  $G$ 

```

that bridges the ASDG and the fuzzer by fully leveraging the rich knowledge encoded in the graph.

Given a device driver δ as the fuzzing target, we first construct an ASDG for δ . Based on the state machine of δ as described by ASDG, we construct an iocltls dependency graph

$\mathcal{G}_\delta$ which includes the iocltls $\{a | a \in \mathcal{G}_\delta\}$. The graph $\mathcal{G}_\delta$ can be further abstracted into a directed cyclic graph with at least one root node r . We aim to synthesize a set of fuzz drivers $\mathbb{D} = \{fd_{a_n} | a_n \in \mathcal{G}_\delta\}$, where a_n is the target ioclt. Each fd_{a_n} has at least one root node and one target ioclt node, and may include additional iocltls necessary to reach a_n through state transitions.

Two key aspects guide synthesis:

- **Synthesis order:** to maximize reuse of runtime knowledge, we prioritize iocltls whose dependencies have been sufficiently explored as targets for synthesizing fuzz drivers, ensuring that updated state knowledge in ASDG can dynamically guide the synthesis process.
- **Semantic guidance:** We leverage the rich content of ASDG (including state transitions, data flows, and parameter constraints) to prompt LLMs in synthesizing semantically valid and state-aware fuzzing drivers while reducing the hallucinations.

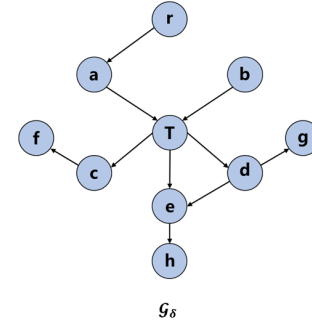


Fig. 3. The dependency graph $\mathcal{G}_\delta$ of iocltls involved in the device driver δ

We first use Fig. 3 as an example to explain the importance of the fuzz drivers synthesis order. To successfully call the target ioclt $a_{\text{target}} = T$, we must first successfully call the preceding iocltls, namely r and a , activating the driver state on which $a_{\text{target}} = T$ depends. Fortunately, after fuzzing fd_a , the valid inputs for r and a are collected and fed into the ASDG (via In-Loop Knowledge Refinement). Serving as state information, these valid inputs will subsequently facilitate the synthesis of high-quality fuzz drivers for $a_{\text{target}} = T$.

The formalization of the synthesis order strategy is detailed in "Nodes Ordering Based on the Priority of Basic Block Counting Weighted Paths" (Algorithm. 2).

First, we collect all valid iocltls call sequences $\mathcal{P}$ target to form $\mathcal{G}_\delta$, i.e., corresponding to step 3 in the Algorithm 2, which can be implemented using DFS. Subsequently, in step 8, we calculate the total number of basic blocks b_{bcP} involving each ioclt in $\mathcal{P}$. In step 10, we sort the iocltls sequences list $\mathbb{L}$ in descending order based on the b_{bcP} to each $\mathcal{P}$ in $\mathbb{L}$. Finally, we traverse all iocltls in list $\mathbb{L}$.

These ordered iocltls will serve as target iocltls and will be used in sequence to synthesize the corresponding fuzz driver.

After prioritizing the target iocltls, we focus on synthesizing the fuzz drivers for target iocltls. A fuzz driver fd_{a_n} targeting ioclt a_n consists of an ordered sequence $\mathcal{A}_{a_n}^{fd} =$

Algorithm 2 Nodes Ordering Based on the Priority of Basic Block Counting Weighted Paths

Require: The ioctl dependency graph $\mathcal{G}_\delta$, the root nodes set $\mathbb{R}$.

Ensure: A node queue $\mathcal{Q}$.

- 1: Let $\mathbb{L} = \emptyset$ be an ioctl sequences list, $\mathcal{Q} = \emptyset$ be an ioctl queue.
 - 2: **for** each root node $r \in \mathbb{R}$. **do**
 - 3: **while** there exists an acyclic path $\mathcal{P}$ without duplicates in $\mathcal{G}_\delta$ rooted at r . **do**
 - 4: $\mathbb{L} = \mathbb{L} \cup \{\mathcal{P}\}$.
 - 5: **end while**
 - 6: **end for**
 - 7: **for** each ioctls sequence $\mathcal{P} \in \mathbb{L}$. **do**
 - 8: Calculate the total number of basic blocks $bbc_{\mathcal{P}}$ involved in all ioctls in the $\mathcal{P}$.
 - 9: **end for**
 - 10: Sort the $\mathbb{L}$ in descending order based on $bbc_{\mathcal{P}}$ of its elements.
 - 11: **for** each ioctls sequence $\mathcal{P} \in \mathbb{L}$. **do**
 - 12: **for** each ioctl $a \in \mathcal{P}$. **do**
 - 13: **if** $a \notin \mathcal{Q}$. **then**
 - 14: Add a to $\mathcal{Q}$
 - 15: **end if**
 - 16: **end for**
 - 17: **end for**
 - 18: Return ioctls queue $\mathcal{Q}$.
-

$\{a_1, a_2, \dots, a_n\}$, where the last ioctl a_n is the target ioctl $a_{target} = a_n$. The preceding calls $\{a_1, \dots, a_{n-1}\}$ form a prerequisite initialization chain, guiding the fuzz driver fd_{a_n} to successfully invoke the target ioctl a_n .

Prior LLM-based fuzz driver synthesis approaches [30], [31] treat all ioctls uniformly, providing no distinction between ioctl $a_{target} = a_n$ and its prerequisite ioctls $\{a_1, \dots, a_{n-1}\}$. This lack of differentiation leads to the LLMs losing guidance towards the target ioctl $a_{target} = a_n$ during fuzz driver synthesis. When fuzzing the synthesized fuzz drivers, fuzzer will struggle to successfully call the prerequisite ioctls $\{a_1, \dots, a_{n-1}\}$, preventing the device from reaching the necessary state to trigger the target ioctl $a_{target} = a_n$.

To address this, we propose a novel state-aware fuzz driver synthesis. Specifically, we perform a fine-grained partitioning of the ordered list of ioctls $\mathcal{A}_{a_n}^{fd} = \{a_1, a_2, \dots, a_n\}$ into two parts: state aware part(SAP) $\{a_1, \dots, a_{n-1}\}$ and target part(TP) a_n . When synthesizing the fd_{a_n} , for ioctl $a_i \in SAP, 1 \leq i \leq n-1$, ASDG will not only provide the LLM with information such as the input parameter structure and header files required by a_i , but also provide fine-grained knowledge for certain members of the structure, such as usage conditions, possible values, and even some typical values from previous fuzzing processes. During fuzzing, the seed exploration space of the synthesized fuzz driver fd_{a_n} that incorporates this knowledge will be significantly compressed. As a result, fd_{a_n}

can invoke the ioctls in SAP more quickly, and the target ioctl $a_{target} = a_n$ can be sufficiently fuzzed. Note that the SAP here serves the target ioctl a_n to enable the device driver δ to reach the state required for a successful invocation of a_n as quickly as possible, thereby achieving sufficient fuzzing of a_n . Fig. 4 below shows the composition of the fuzz driver fd_T with the target ioctl mentioned above as T , where the orange part represents SAP, and the blue part represents TP.



Fig. 4. The code information empowered by ASDG.

Finally, Algorithm 3 below details the state-aware fuzz driver synthesis method based on a multi-agent system driven by LLM. This algorithm consists of four agents. Agent $LLM_{synthesis}$ is responsible for aggregating code information provided by ASDG and synthesizing candidate fuzz driver fd_{a_n} . Subsequently, agent LLM_{refine} performs syntax and rule checks on the fd_{a_n} . The refined fd_{a_n} is then handed over to agent $LLM_{compile}$ for compilation checking. If compilation fails, agent LLM_{fix} is introduced to fix the issues exposed during the compilation of fd_{a_n} . The fuzz driver $fd_{a_{target}} = fd_{a_n}$ that passes compilation is finally output and stored.

2) *Coverage-distance Collaborative Seed Scheduling Exploration*: Coverage-guided and directional distance-guided approaches are two different fuzz driver seed scheduling strategies. The former aims to improve the code coverage to discover more vulnerabilities. The latter aims to trigger one or multiple target code lines as much as possible. In the graph $\mathcal{G}_\delta$, on one hand, the successful invocation of an ioctl depends on the successful invocation of its predecessor ioctls. On the other hand, this dependency is typically implemented by a ioctl explicitly checking whether the predecessor ioctl has triggered a code line indicating the critical state transition. Intuitively, successfully invoking a deeper ioctl in the $\mathcal{G}_\delta$ means having more opportunities to cover more code. Therefore, if we set a code line in a_{target} corresponding to the critical state transition as the target for directed fuzzing, the directed fuzzing will help us successfully invoke a_{target} and establish the necessary preconditions for invoking all subsequent ioctls that depend on a_{target} , thereby improving the code coverage for these deeper ioctls.

From the above observations, we propose a coverage-distance collaborative seed scheduling mechanism based on the growth rate of metrics. Note that the **coverage-guided strategy** maximizes the code coverage, while **distance-guided strategy** focuses on triggering critical state transitions.

Given a fuzz driver $fd_{a_{target}}$, the energy $energy(s)$ of seed s is primarily composed of two parts: coverage-oriented energy $energy_{cov}(s)$ and distance-oriented energy $energy_{dis}(s)$. $energy_{cov}$ follows the seed energy allocation strategy used in AFL++ [32]. And, inspired by He et al. [33], we propose the path prefix code coverage (PPC) PPC_{cov} , which measures

Algorithm 3 State-aware Fuzz Driver Synthesis

Input: A target ioctl a_{target} , the ioctl dependency graph $\mathcal{G}_\delta$.

Output: A fuzz driver $fd_{a_{target}}$.

- 1: Calculate all reverse reachable paths $\mathbb{P} = \{\mathcal{A}_{a_n}^{fd} | a_{target} = a_n\}$ of a_{target} in the $\mathcal{G}_\delta$.
 - 2: Let $\mathcal{A}_{a_n}^{fd} \leftarrow \mathbb{P}$, $\mathcal{A}_{a_n}^{fd} = \{a_0, \dots, a_{n-1}\} || \{a_n\}$.
 - 3: Get $SAP_{a_{n-1}} \leftarrow Query_{ASDG}(\{a_0, \dots, a_{n-1}\})$, $TP_{a_n} \leftarrow Query_{ASDG}(\{a_n\})$.
 - 4: Synthesize a candidate fuzz driver $fd_{a_n} \leftarrow LLM_{synthesis}(SAP_{a_{n-1}}, TP_{a_n})$.
 - 5: Get a refined fuzz driver $fd_{a_n} \leftarrow LLM_{refine}(fd_{a_n})$.
 - 6: **while** Fuzz driver fd_{a_n} compilation is False, Log $\leftarrow LLM_{compile}(fd_{a_n})$. **do**
 - 7: Get a fixed fuzz driver $fd_{a_n} \leftarrow LLM_{fix}(fd_{a_n}, \text{Log})$.
 - 8: **end while**
 - 9: Return the fuzz driver $fd_{a_{target}} = fd_{a_n}$.
-

the proportion of basic blocks involved in the path prefix of target block bb_{target} that covered by a seed s . Note that the target basic block refers to the basic block containing the code line that triggers the critical state transition. Based on PPC, the Equation 1 provides a novel distance-oriented energy $energy_{dis}$ allocation strategy:

$$energy_{dis}(s) = \iota_{a_{target}} \frac{PPC_{cov}(s)}{dis_{min}(s, bb_{target}) + \epsilon} \quad (1)$$

where distance metric $dis_{min}(s, bb_{target})$ represents the minimum distance from the seed s to the target basic block bb_{target} based on the seed trace, and coefficient $\iota_{a_{target}}$ is an amplification factor that requires further discussion. Explicitly, if a node a is closer to the root node r of $\mathcal{G}_\delta$, or if the node a has a higher out-degree, we consider that a successful invocation of this node is more important for fuzzing subsequent nodes in $\mathcal{G}_\delta$. Therefore, when fuzzing this node, the distance-oriented energy of the seed s should be amplified by the factor ι . The Equation 2 provides the formal definition of the coefficients $\iota_{a_{target}}$. For $\forall a \in \mathcal{G}_\delta$, we first obtain the importance I_a as follows:

$$I_a = \beta(D_{\mathcal{G}_\delta} - dis_{min}^{r,a}) + (1 - \beta)degree_{out}^a,$$

where $D_{\mathcal{G}_\delta}$ denotes the depth of the graph $\mathcal{G}_\delta$, $dis_{min}^{r,a}$ denotes the minimum distance between root node r and node a , $degree_{out}^a$ denotes the out degree of the node a , and constant coefficient β is usually used to adjust the weight of the shortest distance of node a and its out-degree. After using softmax to normalize the importance I_a , we can obtain the amplification factor $\iota_{a_{target}}$:

$$\iota_{a_{target}} = 1 + \frac{e^{I_{a_{target}}}}{\sum_{a \in \mathcal{G}_\delta} e^{I_a}} \quad (2)$$

Recall that coverage-distance collaborative seed scheduling mechanism consists of two energy-weighted components:

$$energy(s) = \alpha * energy_{cov}(s) + (1 - \alpha) * energy_{dis}(s).$$

We expect the coefficient α to be able to adaptively adjust to balance the dominance of the two orientations over the seed energy $energy(s)$, and we will discuss this aspect in more detail here.

a) *coverage-guided metric*: Suppose that during fuzzing on a given fuzzing driver fd_{a_n} , we will observe T fuzzing states at fixed time intervals Δt , and the ordered list of ioctls involved in fd_{a_n} is $\mathcal{A}_{a_n}^{fd} = \{a_1, a_2, \dots, a_n\}$. Based on the above content, we introduce the first observation metric, the coverage-guided metric: $\mathcal{C}^{S_i}$, to weight the coverage-oriented energy $energy_{dis}(s)$ for the set of mutated seeds S_i within the time period $\Delta t_i = [t_i, t_{i+1})$. Specifically, the metric $\mathcal{C}^{S_i}$ within the time period Δt_i is obtained by:

$$\mathcal{C}^{S_i} = \frac{\Delta Cov(fd_{a_n}, \Delta t_{i-1})}{TotalCov(\mathcal{A}_{a_n}^{fd})},$$

where the function ΔCov is used to count the increment of basic blocks covered by fd_{a_n} within a time period Δt_{i-1} , and function $TotalCov$ is used to count the total number of basic blocks that fd_{a_n} could theoretically cover.

The metric $\mathcal{C}^{S_i}$ reflects the increase in coverage rate of fd during the previous time period $\Delta t_{i-1} = [t_{i-1}, t_i)$. By analogy, we can obtain the metric set $\mathbb{C}^i = \{\mathcal{C}^{S_j} \mid 0 \leq j < i\}$ before time t_i .

To eliminate the influence of dimensionality, we apply z-score normalization for further processing to obtain the normalized coverage increment metric $\mathcal{C}_{normal}^{S_i} = \frac{\mathcal{C}^{S_i} - \mu_{\mathbb{C}^i}}{\sigma_{\mathbb{C}^i} + \epsilon}$, where μ is used to calculate the mean, σ is used to calculate the standard deviation, and ϵ denotes a very small value used to enhance numerical stability.

b) *distance-guided metric*: Similarly, the second observation metric, the distance-guided metric: $\mathcal{D}^{S_i}$, to weight the distance-oriented energy $energy_{dis}(s)$ for the set of mutated seeds S_i within the time period $\Delta t_i = [t_i, t_{i+1})$. Specifically, the metric: $\mathcal{D}^{S_i}$ within the time period Δt_i is obtained by:

$$\mathcal{D}^{S_i} = \frac{\Delta PPC_{cov}(bb_{target}, \Delta t_{i-1})}{(1 + dis_{min}(bb_{target}, t_i)) PPC_{total}(bb_{target})},$$

where distance metric $dis_{min}(bb_{target}, t_i)$ represents the minimum distance from the seed traces to the target basic block bb_{target} in the previous time period $\Delta t_{i-1} = [t_i, t_{i+1})$, $\Delta PPC_{cov}(bb_{target}, \Delta t_{i-1})$ represents the increment of path prefix code basic blocks of bb_{target} triggered in the previous time period Δt_{i-1} , and $PPC_{total}(bb_{target})$ represents the total number of path prefix code basic blocks involved by bb_{target} .

The metric $\mathcal{D}^{S_i}$ reflects the improvement magnitude of directed fuzzing on fd during the previous time period $\Delta t_{i-1} = [t_{i-1}, t_i)$.

By analogy, we can obtain the metric set $\mathbb{D}^i = \{\mathcal{D}^{S_j} \mid 0 \leq j < i\}$ before time t_i . After applying z-score normalization to $\mathcal{D}^{S_i}$, we obtain the normalized direction increment metric $\mathcal{D}_{normal}^{S_i} = \frac{\mathcal{D}^{S_i} - \mu_{\mathbb{D}^i}}{\sigma_{\mathbb{D}^i} + \epsilon}$. After normalizing the above two

metrics: $C_{normal}^{S_i}$ and $D_{normal}^{S_i}$ using softmax, we obtain the desired coefficient α :

$$\alpha = W_C^i = \frac{e^{C_{normal}^{S_i}}}{e^{C_{normal}^{S_i}} + e^{D_{normal}^{S_i}}},$$

$$1 - \alpha = W_D^i = \frac{e^{D_{normal}^{S_i}}}{e^{C_{normal}^{S_i}} + e^{D_{normal}^{S_i}}}.$$

Finally, in Equation 3, we present the seed energy calculation method for the coverage-distance collaborative seed scheduling mechanism.

$$energy(s_i) = W_C^i energy_{cov}(s_i) + W_D^i energy_{dis}(s_i). \quad (3)$$

3) *In-Loop ASDG Knowledge Refinement*: The construction of ASDG entirely depends on the source code of device drivers. This leads to ASDG having difficulty to accurately infer the potential value ranges or constraints of each member of the input structure for all iocltls. When synthesis a fuzz driver fd_{a_n} , ASDG can only provide limited knowledge for the iocltls in SAP $\{a_1, \dots, a_{n-1}\}$. As a result, fuzzing the fd_{a_n} will still require considerable effort in iocltls $\{a_1, \dots, a_{n-1}\}$. To solve this problem, we propose the in-loop ASDG knowledge refinement mechanism.

In the previous chapter, we proposed the coverage-distance collaborative seed scheduling mechanism, which features a distance-oriented seed scheduling strategy. When we use this mechanism for fuzzing fuzz driver f_{a_n} , once certain seeds S directly triggers the target basic block of a_n , it means that all iocltls in the sequence $\mathcal{A}_{a_n}^{fd}$ have been successfully invoked. Subsequently, we replay these seeds S on f_{a_n} based on the Strace [34] to obtain valid inputs $\{v_{a_1}, \dots, v_{a_n}\}$ for iocltls $\{a_1, \dots, a_n\}$. These valid inputs $\{v_{a_1}, \dots, v_{a_n}\}$ will be used to update the ASDG as typical usages of the structures' members corresponding to these iocltls. When synthesis the fuzz driver $f_{a_{n+1}}$ for the subsequent target iocltls a_{n+1} , the valid inputs $\{v_{a_1}, \dots, v_{a_n}\}$ will be provided to the agent $LLM_{synthesis}$ as the knowledge of SAP $\{a_1, \dots, a_n\}$ to improve the quality of $f_{a_{n+1}}$.

C. Knowledge-Guided Adaptive Exploitation

Knowledge-Guided Adaptive Exploitation aims to uncover potential high-level/deep vulnerability types in device drivers by applying two mutation strategies on the synthesized fuzz drivers. Here we provide a detailed description of the design aspects and functionalities of these two mutation strategies.

1) *Adaptive IOCTL Scheduling Mutation*: During the fuzzing phase, we can collect the respective coverage for different iocltls. At any given moment, the coverage of different iocltls usually implies the current sufficiency of fuzzing for that particular iocltl. Iocltls that have been sufficiently fuzzed should consume fewer computational resources in subsequent fuzzing rounds. To dynamically allocate computational resources to iocltls that require more thorough fuzzing, we propose adaptive iocltl scheduling mutation.

We use a metric called scheduling energy for each iocltl to allocate computational resources. Given an iocltl a , its

scheduling energy $energy_{schedule}$ within the time period $\Delta t_i = [t_i, t_{i+1})$ is given by Equation 4.

$$energy_{schedule}(a) = \frac{1 - \frac{\Delta Cov(a, \Delta t_{i-1})}{TotalCov(a)}}{\gamma |\{fd : a \in fd\}|}, \quad (4)$$

where γ is a constant factor. Intuitively, when an iocltl has higher coverage and is included in the fuzz driver more often, its scheduling energy becomes smaller. We use the scheduling energy of iocltls to sample the iocltls, thereby influencing the frequency of iocltl calls.

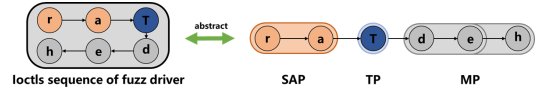


Fig. 5. The dependency graph $\mathcal{G}_\delta$ of iocltls involved in the device driver δ

In the adaptive iocltl scheduling mutation process, we need to clarify two objectives:

- First, we must ensure that SAP, as a state-aware precondition, does not degrade and lose its assigned functions of ensuring that fuzzing can activate target iocltl. In other words, it must be ensured that SAP is replaced with a SAP compatible with target iocltl, or that it is not modified.
- Second, the purpose of behaviors of mutation is centered on the target iocltl, and fully exploit the behaviors that the target iocltl may trigger.

To achieve this, we attempted to abstract the iocltls sequence of fuzz driver into three parts: SAP, TP and mutation part (MP), as detailed in the Fig. 5.

The adaptive iocltl scheduling mutation includes three different strategies:

- **Insertion**: Given a fuzz driver $fd_{a_{target}}$, obtain a batch of iocltls rely on the target iocltl a_{target} from ASDG. Sample an iocltl from the batch of iocltls based on their scheduling energy and insert it into the $fd_{a_{target}}$.
- **Replacement**: Given a fuzz driver $fd_{a_{target}}$, sample an iocltl a_{low} with low energy from MT according to scheduling energy. Sample an iocltl a_{high} with high energy from a batch of iocltls rely on the target iocltl a_{target} . Finally, mutate the fuzz driver $fd_{a_{target}}$ by replacing a_{low} with a_{high} .
- **Parallelism**: Given a fuzz driver $fd_{a_{target}}$, we prompt LLMs to rewrite the invocation methods of some iocltls of $fd_{a_{target}}$ in parallel. The decision on which iocltls' invocation methods to change is made autonomously by the LLMs. Note that this mutation strategy is designed for highly parallel scenarios such as device drivers.

The formalization of adaptive iocltl scheduling mutation is detailed in Algorithm 4.

2) *Knowledge-guided Mutation*: Knowledge-guided mutation aims to enhance the LLMs' ability to understand code vulnerabilities by injecting expert knowledge related to vulnerabilities, guiding the LLMs to generate or mutate fuzz

Algorithm 4 Adaptive IOCTL Scheduling Mutation

Input: A fuzz driver $fd_{a_{target}}$ need to be mutated, the ioctl dependency graph $\mathcal{G}_\delta$.

Output: A mutated fuzz driver $fd_{a_{target}}^{mutated}$.

```
1: Compute all reachable paths  $\mathbb{P} = \{\mathcal{A}_{a_n, a_{leaf}}^{fd} | a_{target} = a_n\}$  of  $a_{target}$  in  $\mathcal{G}_\delta$ .
2: Let  $count_{mutation} \leftarrow 3$ ;
3: while  $count_{mutation} \geq 1$ . do
4:   Let  $mutator \leftarrow Ins, Rep, Par$ .
5:   if  $mutator : Ins$ . then
6:     Sample  $MP_{a_n} \leftarrow \mathbb{P}$  based on the scheduling energy of the successor node  $a_{n+1}$  of  $a_{target} = a_n$ .
7:     Mutate a candidate  $fd_{a_{target}}^{mutated} \leftarrow LLM_{Ins}(fd_{a_{target}}, MP_{a_n})$ .
8:   end if
9:   if  $mutator : Rep$  and  $MP_{a_n} \neq \emptyset$ ,  $MP_{a_n} \leftarrow fd_{a_{target}}$ . then
10:    Sample  $a_{low} \leftarrow MP_{a_n}$ .
11:    Sample  $MP_{a_n}^{new} \leftarrow \mathbb{P}$ 
12:    Mutate a candidate  $fd_{a_{target}}^{mutated} \leftarrow LLM_{Rep}(fd_{a_{target}}, a_{low}, MP_{a_n}^{new})$ .
13:   end if
14:   if  $mutator : Par$ . then
15:    Mutate a candidate  $fd_{a_{target}}^{mutated} \leftarrow LLM_{Par}(fd_{a_{target}})$ .
16:   end if
17:   Refine  $fd_{a_{target}}^{mutated} \leftarrow LLM_{refine}(fd_{a_{target}}^{mutated})$ .
18:   while  $fd_{a_{target}}^{mutated}$  compilation is False, Log  $\leftarrow LLM_{compile}(fd_{a_{target}}^{mutated})$ . do
19:     Get a fixed fuzz driver  $fd_{a_{target}}^{mutated} \leftarrow LLM_{fix}(fd_{a_{target}}^{mutated}, Log)$ .
20:     if  $fd_{a_{target}}^{mutated}$  has new code coverage. then
21:       Return the mutated fuzz driver  $fd_{a_{target}}^{mutated}$ .
22:     end if
23:      $fd_{a_{target}} = fd_{a_{target}}^{mutated}$ .
24:   end while
25: end while
26: Return False.
```

drivers. Specifically, structured knowledge such as known vulnerability patterns, defect types (e.g., buffer overflows, null pointer dereferences, memory leaks), common unsafe function calls, and remediation suggestions is provided as prompts. This enables the LLM to generate fuzz drivers that are semantically sound, achieve broader path coverage, and are more likely to expose potential defects, or to perform targeted mutations on existing drivers, thereby improving the effectiveness of fuzz testing and the efficiency of vulnerability detection.

V. IMPLEMENTATION

The KG-SFUZZ framework consists of two key components: the *ASDG Constructor* and the *Fuzzing Engine*. This

section presents the detailed implementation of each.

A. ASDG Constructor

Our prototype builds an ASDG for kernel device drivers to support state-aware fuzzing over `ioctl` interfaces. The pipeline has three main stages: (1) basic fact extraction, (2) LLM-based semantic inference.

Basic Fact Extraction. We first analyze driver source code with Clang LibTooling [35] to obtain precise AST-level information. From each compilation unit, we extract function prototypes, macro and type declarations, structure nesting relations, direct call relations, and argument type hierarchies (including nested structure fields). These *Basic Facts* form a code-grounded layer that is free of hallucination and later constrain all LLM reasoning.

Direct call edges are obtained directly from previous step. To recover indirect calls that heavily appear in kernel drivers, we integrate DeepType [36], a Strong Multi-Layer Type Analysis (SMLTA)-based framework for C. DeepType models function pointers using multi-layer types (function signature plus surrounding composite types) and enforces strict whole-type matching to determine valid indirect call targets. By resolving information-flow-based relations between multi-layer types and conservatively handling ambiguous types, DeepType augments the call graph with accurate indirect edges, yielding a connected and precise call graph for later reasoning.

LLM-Based Semantic Inference. Given the refined call graph and Basic Facts, we apply an LLM-based reasoning framework constructed atop Qwen3-235B-A22B [37] to infer higher-level semantics needed by fuzzing. First, we refine parameter types whose static types are overly generic (e.g., inferring that an `unsigned long` actually encodes a pointer to a specific kernel structure) by collecting their usage contexts along the call chain and querying the LLM under the constraints of Basic Facts. Second, we derive parameter value ranges from branch conditions and constants (e.g., bounds on `ioctl` numbers), combining syntactic checks with LLM reasoning about symbolic comparisons and enumeration semantics. Finally, we summarize each `ioctl` API's behavior and let the LLM infer (1) state-transition relations (ordering constraints between APIs) and (2) parameter-dependency edges (data-flow relations such as "set-before-get"). All reasoning is performed over Basic Facts and the augmented call graph, which both limits hallucinations and guides the LLM toward code-consistent conclusions. The resulting ASDG serves as the core input to our state-aware fuzzing engine, enabling it to focus on valid state transitions and meaningful parameter interactions.

B. Fuzzing Engine

The KG-SFUZZ Fuzzing Engine integrates knowledge from the ASDG into the fuzzing pipeline through three key mechanisms: state-aware fuzz driver generation & mutation, distance calculation & instrumentation, and a Coverage-Distance Collaborative Seed Scheduler.

State-Aware Fuzz Driver Synthesis & Mutation. We implement a multi-agent fuzz driver synthesizer and mutator based on the CrewAI [38] framework and powered by Qwen3-235B-A22B. Each agent is assigned a specific task in the fuzz driver synthesis pipeline such as fuzz driver synthesis, compilation handling, or error repair. They collaborate under the guidance of the ASDG to iteratively construct and refine system call sequences. The agents ensure that each transition adheres to prerequisite states, parameter constraints, and valid data flows. This enables KG-SFuzz to synthesize high-quality, semantically consistent fuzz drivers that reliably reach deep execution paths.

Distance Calculation & Instrumentation. To support goal-directed exploration, we extend existing coverage measurement tools with state-aware distance tracking. We modify the path-prefix distance calculator from WAFLGo [39] to compute the shortest control-flow distance from each basic block to a target state transition site, such as a state variable update or a critical resource allocation. We further adapt KCOV and LLVM Sancov [40] to insert instrumentation at each related Basic Block (BBL), enabling runtime feedback on how close a seed is to triggering the target state transition.

Coverage-Distance Collaborative Seed Scheduler. Building on this, we enhance AFLGo’s [41] seed scheduling mechanism to implement the Coverage-Distance Collaborative Seed Scheduler. The scheduler maintains two parallel strategies: C-Fuzz favors seeds that increase code coverage, while D-Fuzz prioritizes seeds that reduce control-flow distance to unexplored state transitions. A dynamic weighting mechanism balances both signals based on current exploration progress, ensuring that fuzzing achieves both broad coverage and systematic state advancement. When a new control-flow path successfully triggers a target state transition, KG-SFuzz initiates a replay phase using Strace to capture the sequence of system calls and extract runtime values of key input parameters such as resource indices, buffer sizes, and flag combinations that enabled the transition. These empirically validated values are fed back into the ASDG module and associated with the corresponding transition edge, enriching the ASDG with semantic constraints learned from successful executions. This in-loop feedback enables KG-SFuzz to generate more effective fuzz drivers by leveraging historically successful input patterns across diverse execution contexts.

VI. EVALUATION

We evaluate KG-SFuzz against the vanilla Syzkaller (commit 65a0eec) as the baseline and two state-of-the-art kernel fuzzers: StateFuzz [18] and SyzDescribe [9]. To ensure fair comparison, we have adapted all their toolchains to work with LLVM-15 and Linux kernel 6.7. Our empirical evaluation aims to answer the following research questions:

- RQ1: How expressive and accurate is the ASDG in capturing state transitions?
- RQ2: Can KG-SFuzz achieve higher code coverage than other existing approaches?

- RQ3: Can KG-SFuzz discover vulnerabilities in Linux drivers?

A. Evaluation Setup

We conduct fuzzing experiments on Linux device drivers in two environments. The first environment uses the upstream Linux kernel v6.7 running in QEMU (qemu-system-x86_64). The second environment uses the Linux 5.15 kernel on a Samsung Galaxy A54, a physical Android device.

In the first environment, we run the kernel in QEMU on a server equipped with 192 Intel Xeon PLATINUM 8568Y+ CPU cores and 1 TB RAM, hosted on Ubuntu 24.04 LTS. For each target driver, we deploy two VMs, each with two vCPUs. Each fuzzing campaign runs for 24 hours and is repeated for three independent trials. Both offline preprocessing and online execution use the same kernel configuration as the upstream Syzbot. Additionally, we correct device node names in SyzDescribe and update StateFuzz’s syscall specifications to match the latest syzkaller, ensuring accurate and up-to-date interface modeling.

In the second environment, we perform testing on the physical Galaxy A54 device. Due to compatibility issues in porting StateFuzz and SyzDescribe to this platform, we compare KG-SFuzz only against Syzkaller. Each driver is also fuzzed for 24 hours over three rounds under identical settings.

In both environments, we instrument the kernel using LLVM-15 in conjunction with KCOV, enabling KCOV and KCOV_ENABLE_COMPARISONS for code coverage collection. To maximize measurement accuracy and minimize overhead, we avoid full-kernel instrumentation and instead restrict instrumentation to source code paths relevant to each target driver. For example, when testing DRM drivers, only files under `drivers/gpu/drm/` and its subdirectories are instrumented recursively. Furthermore, we enable the `no-prune` option in KCOV to prevent the compiler from optimizing out instrumentation during compilation. This ensures that all control-flow edges, especially those along critical state transition paths, are preserved and accurately captured during fuzzing.

B. ASDG Evaluation (RQ1)

To evaluate the correctness of our LLM-generated ASDG, we compare it against a manually curated ioctl dependency graph extracted from a production NPU driver. The manual graph is built by thoroughly inspecting the driver’s initialization routines, graph-setup pipeline, streaming procedures, and teardown logic. It comprises 26 state-transition dependencies among 20 interfaces such as `open()`, `BOOTUP`, and `CLOSE`. Using the same driver, our ASDG generation framework yields 42 dependencies. Table V in appendix enumerates all transitions inferred by ASDG, including the ground-truth-consistent edges and the additional semantic edges.

a) *Coverage of Ground-Truth Dependencies.*: ASDG successfully recovers 23 of them, encompassing the canonical forward transitions and all 11 “any ioctl $\rightarrow$ CLOSE” edges. The three missing edges (S_FORMAT $\rightarrow$ PREPARE, S_FORMAT $\rightarrow$ UNPREPARE, FLUSH $\rightarrow$ STREAMOFF) are hidden behind deeply nested teardown logic and intricate object-lifetime rules, rendering them difficult to infer even with the retrieved Basic Facts.

b) *Additional Dependencies Produced by ASDG.*: ASDG introduces 16 extra transitions. A manual inspection reveals that these edges are largely semantically plausible yet previously undocumented interactions, typically arising from optional buffer-re-synchronisation flows, early-return branches, or implicit state propagation across shared internal structures. Although absent from the manual summary, they do not contradict the driver’s semantics.

c) *Discussion.*: Overall, ASDG yields a semantically rich and largely accurate approximation of the driver’s state-transition structure. The missing transitions are confined to the most complex teardown scenarios, while the additional inferred edges contribute useful semantic coverage rather than mere noise. This correctness profile is advantageous for downstream state-aware analysis tasks, where under-approximation risks overlooking valid execution paths.

C. Code Coverage Evaluation (RQ2)

To evaluate whether KG-SFuzz can explore more code paths than state-of-the-art fuzzers within the same time budget, we compare it against Syzkaller, SyzDescribe, and StateFuzz on four general-purpose drivers from Linux-6.7, and against Syzkaller on four proprietary drivers from the Galaxy A54 kernel. For Syzkaller’s syscall specifications on the Galaxy A54, we extend the official templates, such as `syzkaller/sys/linux/dev_video4linux.txt`, with driver-specific knowledge.

As shown in Tables II and III, KG-SFuzz achieves higher code coverage on average across most target drivers. On Linux-6.7, the average coverage increase over Syzkaller is **4.92%**, the improvement over SyzDescribe is **131.24%**, and the gain relative to StateFuzz is **11.46%**. On the Galaxy A54 platform, the average coverage improvement over Syzkaller under identical time constraints is **32.57%**. These results indicate that the knowledge-guided state exploration employed by KG-SFuzz consistently uncovers deeper driver behaviors.

TABLE II
CODE COVERAGE (#BBL) ON GENERAL LINUX 6.7

Tool	DRM	LOOP	ALSA-control	ALSA-pcm
Syzkaller	5615	100	462	1019
SyzDescribe	2093	321	576	275
StateFuzz	4758	333	602	1081
KG-SFuzz	5770	356	640	784

TABLE III
CODE COVERAGE (#BBL) ON GALAXY A54

Tool	NPU	MFC-Decoder	MFC-Encoder	Scaler
Syzkaller	1918	2791	3274	291
KG-SFuzz	3225	3370	3805	569

D. Vulnerability Detection Evaluation (RQ3)

Table IV presents 15 previously undiscovered kernel vulnerabilities identified during our fuzzing campaign. KG-SFuzz successfully detects all 15 vulnerabilities, including 9 that are eligible for CVE assignment, whereas Syzkaller identifies only two. This stark contrast underscores KG-SFuzz’s superior effectiveness in uncovering deep-seated, complex flaws within kernel drivers.

Case Study. List 1 below is the vulnerable code snippet from CVE-*: In the function `__is_done_for_me()`, `c->index` is used as an index into the `q->bufs[]` array without validation, leading to an Out-of-bounds Read.

```
static bool __is_done_for_me(struct vb_queue *
q,
struct vb_bundle **bundle,
struct vs4l_container_list *c)
{
    if (q->bufs[c->index]->state !=
VB_BUF_STATE_DONE) // use c->index without
check
        return false;
    ...
}
```

Listing 1. A snippet code of CVE-*

After testing, this vulnerability could only be successfully triggered by KG-SFuzz. The reason lies in the fact that the function `__is_done_for_me()` resides within a complex call chain involving multiple state transitions (`open()` $\rightarrow$ `BOOTUP` $\rightarrow$ `S_GRAPH` $\rightarrow$ `S_FORMAT` $\rightarrow$ `STREAM_ON` $\rightarrow$ `DQBUF`); only after `c->index` sequentially undergoes this entire sequence of state transitions can it ultimately be passed to this function. In contrast, the Syzkaller lacks the ability to model deep state dependencies, struggles to construct API invocation sequences that are both lengthy and tightly coupled across multiple states, and thus fails to trigger the vulnerability. This phenomenon fully validates the superiority of KG-SFuzz’s approach of constructing precise state transition graphs based on the ASDG. It not only captures fine-grained state transitions but also effectively drives deep, cross-state vulnerability-triggering paths, significantly enhancing the capability to detect complex kernel vulnerabilities.

VII. LIMITATION AND FUTURE WORK

Despite its advances, KG-SFuzz has several limitations that suggest promising directions for future research.

- 1) While the integration of LLMs with static analysis significantly improves semantic accuracy, the initial construction of the ASDG remains sensitive to LLM hal-

TABLE IV
NEW BUGS DETECTED BY KG-SFUZZ

Crash with new specs	CVE	Syzkaller	SyzDescribe	StateFuzz
Out-of-bounds Read in*	CVE-*	✗	N/A	N/A
NULL Pointer Dereference in*	CVE-*	✗	N/A	N/A
Invalid Pointer Dereference in*	CVE-*	✗	N/A	N/A
Invalid Pointer Dereference in*	CVE-*	✓	N/A	N/A
NULL Pointer Dereference in*	CVE-*	✓	N/A	N/A
Null Pointer Dereference in*	Release by March 2026	✗	N/A	N/A
Unvalidated Input Leads to EL2/Firmware Crash	Release by March 2026	✗	N/A	N/A
Null Pointer Dereference in*	Release by March 2026	✗	N/A	N/A
Null Pointer Dereference in*	Release by March 2026	✗	N/A	N/A
Null Pointer Dereference in*	Under analysis	✗	N/A	N/A
Unable to handle kernel paging request in*	Under analysis	✗	N/A	N/A
Out-of-bounds in *	Under analysis	✗	N/A	N/A
Unable to handle kernel paging request in*	Under analysis	✗	N/A	N/A
Out of memory in*	Under analysis	✗	N/A	N/A
Out of memory in*	Under analysis	✗	✗	✗
Total	15	2	0	0

lucinations. Although our LLM-based reasoning framework mitigates this issue, the process still relies on the underlying LLM’s ability. Future work could explore fine-tuning domain-specific LLMs, incorporating formal specifications, and integrating lightweight symbolic execution to validate and refine inferred ranges, reducing reliance on LLM hallucination and enhancing the precision of state-aware fuzzing.

- 2) KG-SFUZZ relies on AFL’s byte-level input mutation, which operates without semantic understanding of the input structure. This limitation frequently results in invalid or malformed `ioctl` argument payloads, reducing fuzzing efficiency and coverage. Future work could introduce semantic-aware mutation operators guided by the ASDG, incorporating value-range analysis of shared state variables, and integrating lightweight symbolic execution to validate inferred constraints.
- 3) Current typical values are maintained independently across distinct control flow paths, limiting the fuzzer’s ability to generalize valid input semantics across different execution contexts. Future work could introduce value-range metrics for shared state variables to model value distributions more effectively.
- 4) The current implementation assumes access to driver source code, limiting applicability to closed-source or binary-only drivers. Future work could utilize hybrid reverse engineering methods, combining binary lifting, dynamic tracing, and LLM-assisted decompilation, to enable reasoning in black-box scenarios.

In summary, KG-SFUZZ opens a new direction in semantics-aware system testing, but the journey toward fully autonomous, intelligent fuzzing for complex kernel code is ongoing. We believe that combining program analysis and AI-driven reasoning, while addressing these practical and technical limitations, will be central to the next generation of secure software development.

VIII. CONCLUSION

This paper introduces KG-SFUZZ, a knowledge-driven, state-aware fuzzing framework that redefines how fuzzers understand and interact with device drivers. By integrating static program analysis with LLM-based semantic, KG-SFUZZ captures the state transitions, data flows, and parameter constraints that govern driver behavior. This semantic richness enables KG-SFUZZ to generate not just syntactically valid, but semantically meaningful test cases. At the core of our approach lies a closed-loop design: the ASDG is initially constructed through iterative LLM inference grounded in static code facts, then continuously refined using runtime feedback. This dynamic evolution allows KG-SFUZZ to capture triggering parameter values, discover new paths, and adapt its fuzzing strategy over time. Guided by the ASDG, KG-SFUZZ employs a Coverage-Distance Collaborative Seed Scheduler, enabling efficient navigation of complex state paths.

Our evaluation across eight real-world Linux drivers demonstrates that KG-SFUZZ significantly outperforms SOTA fuzzers. On average, KG-SFUZZ outperforms Syzkaller by **4.92%**, SyzDescribe by **131.24%**, and StateFuzz by **11.46%** in code coverage on the Linux-6.7 drivers. For Galaxy A54 drivers, KG-SFUZZ achieves a **32.57%** coverage improvement over Syzkaller. During the fuzzing campaign, KG-SFUZZ uncovers 15 previously unknown bugs, 9 of which are CVE-eligible, many residing in deep, multi-call execution sequences that evade conventional fuzzing.

REFERENCES

- [1] Meng Xu et al. “Precise and scalable detection of double-fetch bugs in OS kernels”. In: *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE. 2018, pp. 661–678.

- [2] Shuaibing Lu, Zhechao Lin, and Ming Zhang. “Kernel Vulnerability Analysis: A Survey”. In: *2019 IEEE Fourth International Conference on Data Science in Cyberspace (DSC)*. 2019, pp. 549–554. DOI: 10.1109/DSC.2019.00089.
- [3] Xiaolong Bai et al. “idea: Static analysis on the security of apple kernel drivers”. In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 2020, pp. 1185–1202.
- [4] Google Inc. *syzkaller - kernel fuzzer*. URL: <https://github.com/google/syzkaller>.
- [5] Jake Corina et al. “Difuze: Interface aware fuzzing for kernel drivers”. In: *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 2017, pp. 2123–2138.
- [6] Hui Peng and Mathias Payer. “{USB fuzz}: A framework for fuzzing {USB} drivers by device emulation”. In: *29th USENIX Security Symposium (USENIX Security 20)*. 2020, pp. 2559–2575.
- [7] Zheyu Ma et al. “Printfuzz: Fuzzing linux drivers via automated virtual device simulation”. In: *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 2022, pp. 404–416.
- [8] Wenjia Zhao et al. “Semantic-informed driver fuzzing without both the hardware devices and the emulators”. In: *Network and Distributed Systems Security (NDSS) Symposium 2022*. 2022.
- [9] Yu Hao et al. “Syzdescribe: Principled, automated, static generation of syscall descriptions for kernel drivers”. In: *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE. 2023, pp. 3262–3278.
- [10] Yilun Wu et al. “DEVFUZZ: automatic device model-guided device driver fuzzing”. In: *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE. 2023, pp. 3246–3261.
- [11] Weiteng Chen et al. “Syzgen++: Dependency inference for augmenting kernel driver fuzzing”. In: (2024), pp. 4661–4677.
- [12] HyungSeok Han and Sang Kil Cha. “Imf: Inferred model-based fuzzer”. In: *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 2017, pp. 2345–2358.
- [13] Shankara Pailoor, Andrew Aday, and Suman Jana. “{MoonShine}: Optimizing {OS} fuzzer seed selection with trace distillation”. In: *27th USENIX Security Symposium (USENIX Security 18)*. 2018, pp. 729–743.
- [14] Hao Sun et al. “Healer: Relation learning guided kernel fuzzing”. In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 2021, pp. 344–358.
- [15] Jiacheng Xu et al. “Mock: optimizing kernel fuzzing mutation with context-aware dependency”. In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2024.
- [16] Kunpeng Jian et al. “Fuzzing for Stateful Protocol Implementations: Are We There Yet?” In: *Theoretical Aspects of Software Engineering: 18th International Symposium, TASE 2024, Guiyang, China, July 29 – August 1, 2024, Proceedings*. Guiyang, China: Springer-Verlag, 2024, pp. 186–204. ISBN: 978-3-031-64625-6. DOI: 10.1007/978-3-031-64626-3_11. URL: https://doi.org/10.1007/978-3-031-64626-3_11.
- [17] Chenyuan Yang, Zijie Zhao, and Lingming Zhang. “Kernelgpt: Enhanced kernel fuzzing via large language models”. In: *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 2025, pp. 560–573.
- [18] Bodong Zhao et al. “{StateFuzz}: System {Call-Based}{State-Aware} linux driver fuzzing”. In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022, pp. 3273–3289.
- [19] Kun Hu et al. “A Survey of Fuzzing Open-Source Operating Systems”. In: *arXiv preprint arXiv:2502.13163* (2025).
- [20] Zhuobo Tan and Hui Lu. “A Systemic Review of Kernel Fuzzing”. In: *Proceedings of the 2020 International Conference on Cyberspace Innovation of Advanced Technologies*. 2020, pp. 283–289.
- [21] Wen Xu et al. “From Collision To Exploitation: Unleashing Use-After-Free Vulnerabilities in Linux Kernel”. In: *ACM* (2015).
- [22] Hang Zhang, Dongdong She, and Zhiyun Qian. “Android Root and its Providers: A Double-Edged Sword”. In: *ACM* (2015).
- [23] Dismas Reinald Apostolis, American, and bunnies. “American Fuzzy Lop”. In: *Dic Press* (2012).
- [24] Wei You et al. “SemFuzz: Semantics-based Automatic Generation of Proof-of-Concept Exploits”. In: *ACM* (2017).
- [25] Ivan Pustogarov, Qian Wu, and David Lie. “Ex-vivo dynamic analysis framework for Android device drivers”. In: *2020 IEEE Symposium on Security and Privacy (SP)*. 2020.
- [26] Weiteng Chen et al. “{KOOBE}: Towards facilitating exploit generation of kernel {Out-Of-Bounds} write vulnerabilities”. In: *29th USENIX security symposium (USENIX security 20)*. 2020, pp. 1093–1110.
- [27] Zekun Shen, Ritik Roongta, and Brendan Dolan-Gavitt. “Drifuzz: Harvesting Bugs in Device Drivers from Golden Seeds”. In: (2022).
- [28] Dae R Jeong et al. “Segfuzz: Segmentizing thread interleaving to discover kernel concurrency bugs through fuzzing”. In: *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE. 2023, pp. 2104–2121.
- [29] Hsin Wei Hung and Ardalan Amiri Sani. “BRF: Fuzzing the eBPF Runtime”. In: *Proceedings of the ACM on Software Engineering* 1.FSE (2024), pp. 1152–1171.
- [30] Yunlong Lyu et al. “Prompt fuzzing for fuzz driver generation”. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2024, pp. 3793–3807.

- [31] Hanxiang Xu et al. “CKGFuzzer: LLM-Based Fuzz Driver Generation Enhanced By Code Knowledge Graph”. In: *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE. 2025, pp. 243–254.
- [32] Andrea Fioraldi et al. “{AFL++}: Combining incremental steps of fuzzing research”. In: *14th USENIX workshop on offensive technologies (WOOT 20)*. 2020.
- [33] Yubo He and Long Liu. “Critical Variable Guided Mutation for Directed Greybox Fuzzing”. In: *2023 IEEE International Conference on Control, Electronics and Computer Technology (ICCECT)*. IEEE. 2023, pp. 1541–1547.
- [34] Strace. URL: <https://strace.io/>.
- [35] *Clang LibTooling*. URL: <https://clang.llvm.org/docs/LibTooling.html>.
- [36] Jonathan Raiman and Olivier Raiman. “DeepType: Multilingual Entity Linking by Neural Type System Evolution”. In: (2018).
- [37] An Yang et al. “Qwen3 Technical Report”. In: *arXiv preprint arXiv:2505.09388* (2025).
- [38] *CrewAI*. URL: <https://github.com/crewai/crewai>.
- [39] Yi Xiang et al. “Critical code guided directed greybox fuzzing for commits”. In: *33rd USENIX Security Symposium (USENIX Security 24)*. 2024.
- [40] *SanitizerCoverage*. URL: <https://clang.llvm.org/docs/SanitizerCoverage.html>.
- [41] Marcel Böhme et al. “Directed Greybox Fuzzing”. In: *Acm SigSAC Conference on Computer Communications Security*. 2017, pp. 2329–2344.

APPENDIX

TABLE V
MANUAL AND ASDG-DERIVED
IOCTL STATE DEPENDENCIES FOR THE NPU DRIVER.

Source → Target	Origin	Classification
INIT → open()	Manual+ASDG	Ground-truth
open() → BOOTUP	Manual+ASDG	Ground-truth
open() → SCHED_PARAM	Manual+ASDG	Ground-truth
open() → PROFILEON	Manual+ASDG	Ground-truth
open() → PROFILEOFF	Manual+ASDG	Ground-truth
open() → G_MAX_FREQ	Manual+ASDG	Ground-truth
BOOTUP → S_GRAPH	Manual+ASDG	Ground-truth
BOOTUP → S_PARAM	Manual+ASDG	Ground-truth
S_GRAPH → S_FORMAT	Manual+ASDG	Ground-truth
S_GRAPH → VERSION	Manual+ASDG	Ground-truth
S_FORMAT → STREAMON	Manual+ASDG	Ground-truth
STREAMON → poll()	Manual+ASDG	Ground-truth
FLUSH → CLOSE	Manual+ASDG	Ground-truth
open() → CLOSE	Manual+ASDG	Ground-truth
BOOTUP → CLOSE	Manual+ASDG	Ground-truth
S_GRAPH → CLOSE	Manual+ASDG	Ground-truth
S_FORMAT → CLOSE	Manual+ASDG	Ground-truth
PREPARE → CLOSE	Manual+ASDG	Ground-truth
UNPREPARE → CLOSE	Manual+ASDG	Ground-truth
QBUF → CLOSE	Manual+ASDG	Ground-truth
DQBUF → CLOSE	Manual+ASDG	Ground-truth
STREAMON → CLOSE	Manual+ASDG	Ground-truth
STREAMOFF → CLOSE	Manual+ASDG	Ground-truth
FLUSH → STREAMOFF	Manual-only	Missing
S_FORMAT → PREPARE	Manual-only	Missing
S_FORMAT → UNPREPARE	Manual-only	Missing
open() → S_GRAPH	ASDG-only	Plausible
open() → S_FORMAT	ASDG-only	Plausible
open() → S_PARAM	ASDG-only	Plausible
open() → VERSION	ASDG-only	Plausible
BOOTUP → S_CTRL	ASDG-only	Plausible
BOOTUP → VERSION	ASDG-only	Plausible
BOOTUP → G_MAX_FREQ	ASDG-only	Plausible
S_GRAPH → S_PARAM	ASDG-only	Plausible
S_GRAPH → PREPARE	ASDG-only	Plausible
S_FORMAT → VERSION	ASDG-only	Plausible
PREPARE → QBUF	ASDG-only	Plausible
STREAMON → poll()	ASDG-only	Plausible
QBUF → DQBUF	ASDG-only	Plausible
DQBUF → QBUF	ASDG-only	Plausible
PROFILEON → PROFILEOFF	ASDG-only	Plausible
S_CTRL → S_CTRL	ASDG-only	Hallucinated